The OMI Tetrahedral QuQuart and Quasigroup CONS Model

Authoritative Architecture for Finite Interpretation, Reversible Continuation, Centroid Reduction, and Receipt-Stable Execution

Status

Canonical integration model.

This document defines the relationship among:

```
RULES.o
FACTS.o
COMBINATORS.o
CLOSURES.o
CONS.o

Q0 source
Q1 notation
Q2 reading
Q3 receipt

CAR
CDR
CONS

Boolean truth-table capacity
Latin-square resolution
quasigroup recovery
polyadic centroid reduction

Omicron
Tetragrammatron
Metatron
validation
receipt
```

The central architecture is:

```
RULES.o + FACTS.o
        ↓
      Q0 source
        ↓
      Q1 notation
        ↓
COMBINATORS.o
        ↓
      Q2 reading
        ↓
 CLOSURES.o
        ↓
      Q3 receipt

CONS.o
=
centroid reducer across Q0-Q3
```

The core doctrine is:

```
The QuQuart has four interpretation vertices.

CONS is not a fifth QuQuart basis state.

CONS is the centroid relation that selects,
interpolates, reduces, and recovers
the four QuQuart coordinates.
```

---

1. The Four-State Interpretive QuQuart

The OMI QuQuart is a deterministic four-state interpretation register:

```
Q0 = source
Q1 = notation
Q2 = reading
Q3 = receipt
```

These are not physical quantum states.

They are typed positions in an interpretation lifecycle.

Q0 — Source

The source is the versioned binary authority from which a reading begins.

In the `.o` model:

```
Q0 = RULES.o + FACTS.o
```

where:

```
RULES.o
supplies the declared source law

FACTS.o
supplies grounded source conditions
```

The source is not merely raw bytes.

It is the relation:

```
declared law
+
grounded fact
```

A source that lacks either law or ground remains incomplete.

Q1 — Notation

Notation is the declared representation through which Q0 becomes addressable.

It may include:

```
bitboard layout
field width
RRGGBBAA vector
hexadecimal row
Omicron plane
F* dialect
address mask
glyph
OMI-Lisp declaration
```

Q1 does not alter the source.

It defines how the source is to be read.

Q2 — Reading

Reading is the active interpretation produced by applying a lawful operation.

In the `.o` model:

```
Q2 = COMBINATORS.o(Q0,Q1)
```

COMBINATORS may:

```
select
project
fold
rotate
compare
mask
compose
interpolate
reduce
```

A reading is still a candidate until it passes closure and validation.

Q3 — Receipt

Receipt is the accepted interpretation commitment.

In the `.o` model:

```
Q3 = CLOSURES.o(Q0,Q1,Q2)
```

CLOSURES determines whether:

```
the source was correctly grounded

the notation was correctly declared

the reading followed an allowed route
```

```
the result can be replayed

the receipt identity remains stable
```

The QuQuart lifecycle is therefore:

```
source
→ notation
→ reading
→ receipt
```

---

2. The Tetrahedral Interpretation Cell

A tetrahedron provides:

```
4 vertices
6 edges
4 triangular faces
1 centroid relation
```

The four QuQuart-bearing `.o` roots occupy the vertices:

```
V0 = RULES.o
V1 = FACTS.o
V2 = COMBINATORS.o
V3 = CLOSURES.o
```

The centroid is:

```
C = CONS.o
```

A useful schematic is:

```
                  CLOSURES.o
                  Q3 receipt
                       ▲
                      /|
                     / |
                    /  |
```

```
          RULES.o --- CONS.o --- COMBINATORS.o
          source law   centroid    reading operation
              \          |          /
               \         |         /
                \        |        /
                   FACTS.o
              grounded source
```

The centroid is not another vertex.

It is the relation that binds the vertices into one memory cell.

Canonical statement:

```
The vertices carry typed interpretation roles.

The centroid carries their combined resolution law.
```

---

3. Why CONS Is the Centroid

Traditional Lisp defines:

```
CONS = (CAR . CDR)
```

In the OMI model:

```
CAR
is the currently carried coordinate

CDR
is the continuation law or resolver

CONS
is the relation produced by applying
the continuation to the carried coordinate
```

Formally:

$$CONS(CAR, CDR) = CDR(CAR)$$

when the CDR is interpreted as a function.

In the quasigroup form:

$$CAR * CDR = CONS$$

The centroid role appears because CONS can relate:

```
one vertex
two vertices
one triangular face
all four vertices
```

depending on the selected reducer profile.

Thus CONS supports:

```
vertex selection
edge resolution
face interpolation
centroid reduction
receipt replay
```

---

4. The Six Tetrahedral Edges

The four vertices define six pairwise relations.

RULES–FACTS

```
Does the grounded fact satisfy the declared source law?
```

RULES–COMBINATORS

```
Is the proposed reading operation permitted by the source law?
```

RULES–CLOSURES

```
Does the closure or receipt satisfy the declared law?
```

FACTS–COMBINATORS

```
Can the operation lawfully apply to the grounded source?
```

FACTS–CLOSURES

```
Does the receipt actually bind the source facts?
```

COMBINATORS–CLOSURES

```
Did the executed reading pass its verification boundary?
```

These six edge predicates may be represented as a six-bit mask:

```
bit 0 = RULES–FACTS
bit 1 = RULES–COMBINATORS
bit 2 = RULES–CLOSURES
bit 3 = FACTS–COMBINATORS
bit 4 = FACTS–CLOSURES
bit 5 = COMBINATORS–CLOSURES
```

A centroid reducer can require any declared subset of these edges.

---

5. The Four Triangular Faces

The tetrahedron also defines four three-vertex surfaces.

RULES / FACTS / COMBINATORS

Candidate-reading construction:

```
law
+
ground
+
operation
→ candidate reading
```

RULES / FACTS / CLOSURES

Source-grounded verification:

```
law
+
ground
+
closure
→ source acceptance predicate
```

RULES / COMBINATORS / CLOSURES

Operation-law verification:

```
law
+
operation
+
closure
→ lawful-route predicate
```

FACTS / COMBINATORS / CLOSURES

Result-grounded verification:

```
ground
+
operation
+
closure
→ result-grounding predicate
```

A complete local centroid acceptance may require:

$$F_0 \wedge F_1 \wedge F_2 \wedge F_3$$

where each F_i is a face-consistency predicate.

This gives the local tetrahedral cell a complete verification structure even before larger governors are invoked.

---

6. The Boolean CONS Ladder

For selector rank n, define the CAR domain:

$$X_n = \{0,1\}^n$$

Therefore:

$$|X_n| = 2^n$$

A Boolean CDR is a function:

$$f : X_n \to \{0,1\}$$

One complete truth table requires:

$$2^n \text{ bits}$$

The total number of distinct Boolean functions is:

$$2^{2^n}$$

These are three different quantities:

```
selector rank
CAR assignment count
CDR function population
```

They must not be collapsed into one exponent.

---

7. Canonical Capacity Table

Rank n	CAR assignments	One CDR truth-table width	Number of possible CDR functions
`-1`	escape/meta	declared envelope	profile-defined
`0`	$2^0 = 1$	1 bit	$2^1 = 2$
`1`	$2^1 = 2$	2 bits	$2^2 = 4$
`2`	$2^2 = 4$	4 bits	$2^4 = 16$
`3`	$2^3 = 8$	8 bits	$2^8 = 256$
`4`	$2^4 = 16$	16 bits	2^{16}

Rank n	CAR assignments	One CDR truth-table width	Number of possible CDR functions
5	$2^5 = 32$	32 bits	2^{32}
6	$2^6 = 64$	64 bits	2^{64}
7	$2^7 = 128$	128 bits	2^{128}
8	$2^8 = 256$	256 bits	2^{256}

The crucial implementation rule is:

```
One selected rank-8 CDR is 256 bits wide.

There are 2^256 possible such CDR values.

The runtime stores the selected 256-bit CDR.

It does not allocate 2^256 CDR objects.
```

---

8. Truth-Table Width Versus Function Population

At rank 8:

```
CAR assignment domain
=
256 byte values
```

One selected CDR is:

```
256 output bits
=
32 bytes
```

Each bit answers:

```
Does this CDR accept or reject
this specific byte assignment?
```

Thus a rank-8 CDR is a predicate over the entire byte field:

$$f : \{0,1\}^8 \to \{0,1\}$$

A 256-bit bitboard stores one such predicate.

Example:

```
typedef struct {
    uint64_t lane[4];
} OmiPredicate256;
```

Lookup:

```
bool omi_predicate_eval(
    const OmiPredicate256 *predicate,
    uint8_t input
) {
    uint8_t lane = input >> 6;
    uint8_t bit  = input & 63u;

    return (
        (predicate->lane[lane] >> bit) & 1u
    ) != 0u;
}
```

---

9. Why Boolean Capacity Alone Is Not Enough

The Boolean ladder tells us:

```
how many inputs exist

how wide one selected function is

how many possible functions exist
```

It does not by itself tell us:

```
how to combine a CAR and CDR reversibly

how to recover a missing coordinate

how to distinguish CAR order from CDR order
```

```
how to select resolver dialects

how to reconcile partial tuples
```

This is where quasigroups and Latin squares enter.

Canonical separation:

```
Boolean theory defines capacity.

Quasigroup theory defines reversible resolution.
```

---

10. Latin Squares as Finite Resolvers

A Latin square of order m contains m symbols arranged so that each symbol occurs exactly once in each row and each column.

This gives:

```
row + column → symbol

row + symbol → column

column + symbol → row
```

A finite quasigroup expresses the same property algebraically.

Let:

$$a * b = c$$

Then the quasigroup property guarantees unique solutions for:

$$a * x = c$$

and:

$$y * b = c$$

Therefore:

```
given CAR and CDR, recover CONS

given CAR and CONS, recover CDR

given CDR and CONS, recover CAR
```

This is the reversible local resolver required by dotted CONS.

---

11. The CONS Triple

The dotted pair:

```
(CAR . CDR)
```

produces a complete quasigroup triple:

```
(CAR, CDR, CONS)
```

where:

$$CONS = CAR * CDR$$

The full law is:

```
CAR * CDR = CONS

CAR \ CONS = CDR

CONS / CDR = CAR
```

The specific symbols `*`, `\`, and `/` are conventional.

The important part is unique recovery.

This means a serialized CONS relation may omit one coordinate when the other two and the resolver profile are known.

---

12. Why Plain XOR Is Insufficient

The current local witness:

```
encode(CAR) XOR encode(CDR)
```

has useful properties:

```
simple

symmetric

detects complement

(0xAA55 . 0x55AA) → 0xFFFF
```

But it also causes structural loss.

Because XOR is commutative:

$$A \oplus B = B \oplus A$$

the pairs:

```
(A . B)
```

and:

```
(B . A)
```

produce the same witness.

It also gives:

$$A \oplus A = 0$$

so:

```
(A . A)
(NULL . NULL)
```

may collapse to the same value.

Therefore the XOR result may be used as:

```
contrast witness
complement witness
local parity witness
```

It must not be used as:

```
unique structural identity
reversible executable encoding
canonical receipt hash
```

---

13. Ordered Quasigroup Encoding

A practical fixed-width quasigroup can be constructed using distinct invertible transforms.

Let:

$$P : V \to V$$

and:

$$Q : V \to V$$

be invertible permutations over a fixed-width bit-vector space V.

Let $k \in V$ be a declared constant.

Define:

$$a * b = P(a) \oplus Q(b) \oplus k$$

Then:

$$b = Q^{-1}(c \oplus P(a) \oplus k)$$

and:

$$a = P^{-1}(c \oplus Q(b) \oplus k)$$

This preserves:

```
bounded width

unique recovery

ordered CAR/CDR roles

algorithmic execution

no stored giant Latin table
```

Because P and Q differ, swapping CAR and CDR generally changes the result.

---

14. Algorithmic Latin Squares

A Latin square does not have to be stored as a two-dimensional matrix.

For a 256-bit carrier, a literal table would be impossible to materialize.

Instead, the resolver is represented by:

```
carrier type

CAR permutation

CDR permutation

symbol permutation

constant or seed

inverse operations
```

The operation itself generates the relevant Latin-square entry.

Canonical rule:

```
The Latin square is a law, not necessarily a stored table.
```

This is what makes enormous conceptual quasigroup orders executable.

---

15. F* as Resolver Family and Dialect

The gauge form:

```
F*
```

can identify a quasigroup profile.

Interpretation:

```
F
resolver family

*
isotopic dialect or orbit offset
```

A Latin-square isotopy applies independent permutations to:

```
rows

columns

symbols
```

Formally:

$$L'(a,b) = \gamma^{-1}\left(L(\alpha(a), \beta(b))\right)$$

where:

```
α
CAR row permutation

β
CDR column permutation

γ
CONS symbol permutation
```

Thus the F* row can select:

```
CAR dialect

CDR dialect

CONS projection dialect
```

while preserving the Latin uniqueness law.

This provides dialect variation without changing the fundamental resolver family.

---

16. The Order-4 QuQuart Latin Resolver

The QuQuart itself has four coordinates:

```
Q0
Q1
Q2
Q3
```

A Latin square of order four can define reversible transitions among them.

A simple cyclic profile is:

*	Q0	Q1	Q2	Q3
Q0	Q0	Q1	Q2	Q3
Q1	Q1	Q2	Q3	Q0
Q2	Q2	Q3	Q0	Q1
Q3	Q3	Q0	Q1	Q2

This profile gives:

```
current QuQuart coordinate
+
operation offset
→
resolved QuQuart coordinate
```

For example:

```
Q0 * Q1 = Q1
Q1 * Q2 = Q3
Q3 * Q1 = Q0
```

Any two coordinates recover the third.

A noncommutative order-4 quasigroup may be selected when CAR/CDR direction must remain more strongly visible.

---

17. The Sixteen QuQuart Reducers

A four-vertex QuQuart has a four-bit selector mask.

Therefore it admits:

$$2^4 = 16$$

possible subset-selection predicates.

Examples:

```
0000
select no vertex

0001
select Q0 source only

0011
select Q0 source and Q1 notation

0101
select Q0 source and Q2 reading

0111
select source, notation, and reading

1000
select Q3 receipt only

1111
select the complete QuQuart
```

These sixteen masks form the local two-ququart or four-bit operator surface.

A selected CDR may therefore serve as:

```
vertex selector

edge selector

face selector

acceptance mask

projection mask

interpolation profile
```

---

18. Polyadic Quasigroup Centroid

A binary quasigroup resolves:

$$CAR * CDR = CONS$$

But the tetrahedral cell contains four vertices.

A polyadic quasigroup can define:

$$F(R, F, C, L) = S$$

where:

```
R = RULES
F = FACTS
C = COMBINATORS
L = CLOSURES
S = CONS centroid
```

The required polyadic recovery property is:

```
given any four of the five coordinates,
recover the missing fifth coordinate
```

Thus:

$$S = F(R, F, C, L)$$

and inverse operations permit:

$$R = F_R^{-1}(S, F, C, L)$$

$$F = F_F^{-1}(R, S, C, L)$$

$$C = F_C^{-1}(R, F, S, L)$$

$$L = F_L^{-1}(R, F, C, S)$$

This gives the tetrahedral centroid an exact computational meaning:

```
CONS is a recovery witness binding
RULES, FACTS, COMBINATORS, and CLOSURES.
```

Again:

```
CONS is the fifth recovery coordinate.

CONS is not Q4.
```

---

19. Local Execution Without Tetragrammatron

The tetrahedral QuQuart cell can operate locally without invoking the full Tetragrammatron.

Local operations include:

```
parse dotted CONS

evaluate one selected CDR

resolve a QuQuart vertex

evaluate tetrahedral edges

evaluate tetrahedral faces

derive a centroid result

construct a local receipt candidate
```

```
replay the same route
```

A local cell may therefore execute:

```
((RULES . FACTS) . (COMBINATORS . CLOSURES))
```

using only:

```
Omicron readiness

OMI-Lisp parser

quasigroup profile

bitboard operations

closure predicates

receipt primitive
```

---

20. When Tetragrammatron Is Required

Tetragrammatron becomes necessary when the computation must govern:

```
the five interpretation governors

27+5 Omicron relation partition

cross-sector relation closure

cross-frame reassociation

5! circulation

local240 placement

boot/runtime/userspace bands

global relation validation
```

A quasigroup does not generally provide associativity.

Therefore:

$$(A * B) * C$$

need not equal:

$$A * (B * C)$$

This is useful because interpretation order can matter.

Tetragrammatron governs when reassociation is lawful.

Canonical distinction:

```
CONS resolves locally.

Tetragrammatron governs larger reassociation and closure.
```

A Moufang-loop-like profile may be used where selected reassociation identities are required without demanding global associativity.

---

21. When Metatron Is Required

Metatron is not required to compute the local result.

Metatron is required when provenance must record:

```
CAR row

CDR column

CONS symbol

Latin-square profile

F* dialect

tetrahedral vertex

tetrahedral edge
```

```
tetrahedral face

FS/GS/RS/US scope

open/sealed polarity

local60

local240

azimuth

hardware face

source and destination relation
```

Canonical distinction:

```
CONS computes or resolves the relation.

Metatron records where and how
that relation was traversed.
```

Metatron may store the quasigroup triple:

```
(CAR,CDR,CONS)
```

or the polyadic tuple:

```
(RULES,FACTS,COMBINATORS,CLOSURES,CONS)
```

without changing the validation status.

---

22. Omicron and Earned Dot Notation

The OMI-Lisp parser requires Omicron to reach readable state before dotted notation is legal.

The sequence is:

```
pre-header

→ pre-language topology

→ SP crossing

→ dot earned

→ dotted CONS declaration becomes readable
```

This creates an important authority boundary.

```
Control-space bytes
do not automatically become executable syntax.
```

The dot is not merely punctuation.

It represents an operation that becomes available only after the runtime crosses from control topology into readable notation.

Canonical statement:

```
Omicron makes the dotted relation reachable.

The parser makes it syntactically valid.

The resolver makes it executable.

Validation and receipt determine acceptance.
```

---

23. Canonical Structural Serialization

An executable CONS relation must preserve:

```
node type

CAR/CDR order

atom type
```

```
field width

nested boundaries

resolver profile

dialect

version
```

A canonical binary form should use explicit tags and lengths.

Example:

```
CONS_TAG
VERSION
PROFILE_ID
CAR_TYPE
CAR_LENGTH
CAR_BYTES
CDR_TYPE
CDR_LENGTH
CDR_BYTES
```

Nested CONS nodes recursively contain the same form.

The receipt commitment should hash the complete canonical serialization, not only a 16-bit XOR witness.

---

24. Suggested 64-Bit Local Bitboard

A compact local execution cell can be represented as:

```
bits  0..3
QuQuart selector mask

bits  4..9
six tetrahedral edge results

bits 10..13
four tetrahedral face results

bits 14..16
Hamming syndrome
```

```
bits 17..18
Delta3 position

bits 19..22
FS/GS/RS/US scope

bits 23..30
Gnomonic azimuth

bits 31..46
CONS local witness

bits 47..62
receipt or continuation fragment

bit 63
accepted flag
```

This is not the only possible packing.

Its purpose is to demonstrate that the complete local model is finite and executable.

---

25. Suggested Runtime Types

```c
typedef enum {
    OMI_Q0_SOURCE   = 0,
    OMI_Q1_NOTATION = 1,
    OMI_Q2_READING  = 2,
    OMI_Q3_RECEIPT  = 3
} OmiQuquartVertex;

typedef struct {
    uint64_t rules;
    uint64_t facts;
    uint64_t combinators;
    uint64_t closures;
} OmiTetrahedralVertices;

typedef struct {
    uint8_t selector4;
    uint8_t edge_mask6;
    uint8_t face_mask4;
```

```
    uint64_t centroid_value;
    uint64_t structural_witness;

    bool accepted;
} OmiConsCentroid;

typedef struct {
    uint64_t car;
    uint64_t cdr;
    uint64_t symbol;

    uint16_t profile_id;
    uint8_t dialect;
} OmiQuasigroupTriple;
```

For rank-8 predicates:

```
typedef struct {
    uint64_t lane[4];
} OmiWord256;
```

---

26. Authoritative Execution Pipeline

```
1. FRAME

   Omicron recognizes the pre-header
   and stages the runtime plane.

2. EARN NOTATION

   Omicron crosses SP.
   Dotted CONS notation becomes reachable.

3. PARSE

   Parse canonical ordered dotted pairs.

4. LOAD SOURCE LAW

   Load RULES.o.

5. LOAD SOURCE GROUND
```

```
   Load FACTS.o.

6. BUILD Q0

   Resolve the grounded source.

7. BUILD Q1

   Generate or select the notation surface.

8. SELECT CDR

   Load one bounded continuation predicate
   or quasigroup resolver profile.

9. RESOLVE Q2

   Apply COMBINATORS.o through the selected CDR.

10. EVALUATE EDGES

    Evaluate the six pairwise tetrahedral relations.

11. EVALUATE FACES

    Evaluate the four triangular consistency surfaces.

12. BUILD Q3 CANDIDATE

    Apply CLOSURES.o to the source,
    notation, reading, edges, and faces.

13. CENTROID REDUCTION

    CONS.o selects and reduces Q0-Q3.

14. RECOVER IF NEEDED

    Use quasigroup or polyadic inverse operations
    to recover a missing bounded coordinate.

15. GOVERN IF NEEDED

    Invoke Tetragrammatron for global closure,
    reassociation, sector governance, or local240.

16. SCRIBE IF NEEDED
```

```
    Invoke Metatron to record incidence,
    traversal, dialect, and projection provenance.

17. VALIDATE

    Accept, reject, or preserve unresolved status.

18. RECEIPT

    Bind source, notation, reading, result,
    resolver profile, and route into a receipt.

19. REPLAY

    Re-run the declared route against the stable source.

20. CONFIRM STABILITY

    The same lawful route must produce
    the same receipt commitment.
```

---

27. Receipt Replay Stability

The authoritative stability property is:

```
same source
same notation
same reading route
same resolver profile
same result
→ same receipt
```

This is receipt replay stability.

It is different from claiming that every rewrite operator is mathematically idempotent.

The safe law is:

$$Receipt(S, N, R, P) = Receipt(S, N, R, P)$$

under deterministic replay.

A stronger acceptance operation may satisfy:

$$Accept(Accept(S, r), r) = Accept(S, r)$$

but this must be tested explicitly.

---

28. Blackboard and Distributed Resolution

The same quasigroup triple can later be used in a blackboard or distributed resolver.

A bounded contribution is:

```
(CAR,CDR,CONS)
```

Any peer, module, or knowledge source holding two coordinates can recover the third, provided it has the same resolver profile.

The merge rules are:

```
one result for each CAR/CDR pair

unique CDR recovery from CAR/CONS

unique CAR recovery from CDR/CONS

same field width

same resolver family

compatible dialect

compatible receipt ancestry
```

This makes resolver fragments:

```
finite

shardable

reversible

merge-checkable
```

```
locally verifiable
```

The algebra does not itself decide application behavior.

---

29. Authority Boundaries

The following distinctions are mandatory.

```
Boolean capacity
does not validate a relation.
```

```
A Latin-square result
does not imply semantic truth.
```

```
Quasigroup recovery
does not imply acceptance.
```

```
CONS reduction
does not automatically produce a receipt.
```

```
Metatron incidence
does not change validation status.
```

```
Tetragrammatron governance
does not replace source authority.
```

```
Projection
does not mutate the source.
```

```
Validation decides.

Receipt records and stabilizes the accepted interpretation.
```

---

30. Compact Canonical Table

Component	Role
`RULES.o`	source law
`FACTS.o`	grounded source
`Q0`	combined source
`Q1`	notation
`COMBINATORS.o`	reading operation
`Q2`	active reading
`CLOSURES.o`	receipt boundary
`Q3`	accepted or candidate receipt
`CAR`	carried selector or value
`CDR`	continuation law or selected function
`CONS.o`	centroid reducer and recovery witness
Boolean ladder	capacity
Latin square	finite reversible table law
Quasigroup	unique recovery operation
Polyadic quasigroup	tetrahedral coordinate recovery
Omicron	runtime and earned notation boundary
Tetragrammatron	global closure and reassociation governor
Metatron	incidence and traversal scribe
Validation	acceptance decision
Receipt	replay-stable commitment

---

31. One-Line Canon

OMI models interpretation as a tetrahedral QuQuart with source, notation, reading, and receipt as four typed vertices; RULES.o and FACTS.o ground Q0, COMBINATORS.o produces Q2 through Q1 notation, CLOSURES.o establishes Q3, and CONS.o acts as the centroid reducer rather than a fifth basis state; the Boolean ladder determines continuation capacity, while Latin-square and quasigroup laws provide finite reversible resolution in which CAR,

CDR, and CONS form a triple from which any missing coordinate can be recovered; Tetragrammatron governs larger closure and reassociation, Metatron records incidence, validation accepts, and receipt confirms deterministic replay stability.

---

32. Final Canon Lock

```
The QuQuart has four vertices:

Q0 source
Q1 notation
Q2 reading
Q3 receipt.
```

```
RULES.o supplies law.

FACTS.o supplies ground.

COMBINATORS.o supplies operation.

CLOSURES.o supplies the receipt boundary.
```

```
CONS.o is the tetrahedral centroid.

CONS is not Q4.
```

```
CAR selects or carries.

CDR continues or resolves.

CONS is their resolved relation.
```

```
At rank n:

CAR has 2^n possible assignments.

One CDR truth table occupies 2^n bits.

There are 2^(2^n) possible CDR functions.
```

```
At rank 8:

CAR selects among 256 byte inputs.

One CDR occupies 256 bits.

There are 2^256 possible CDR predicates.

The runtime stores one selected predicate,
not 2^256 objects.
```

```
Boolean theory tells us how large the space is.

Quasigroup theory tells us how to navigate it reversibly.
```

```
A Latin-square law gives:

CAR + CDR → CONS

CAR + CONS → CDR

CDR + CONS → CAR.
```

```
A polyadic quasigroup binds:

RULES
FACTS
COMBINATORS
CLOSURES
CONS

so a missing coordinate may be recovered
from the remaining bounded coordinates.
```

```
Omicron earns the dot.

OMI-Lisp declares the relation.

CONS resolves locally.

Tetragrammatron governs globally.
```

```
Metatron scribes traversal.

Validation accepts.

Receipt records and confirms replay stability.
```